JADAVPUR UNIVERSITY

188, Raja S.C. Mallick Road, Kolkata – 700032

Department of Computer Science and Engineering, FET.

MASTER OF COMPUTER APPLICATION

1st Year 2nd Semester

OPERATING SYSTEM ASSIGNMENT

Name: Snehasish Sarkar
Roll No.: 002510503033

QUESTION 1: Write a shell script that asks for the values of 2 variables: userv1 and userv2, from the user and takes in any type (real/integer/character). Try to perform the following operations on the variables: (a) add, (b) multiply (c) subtract (d) division. Print appropriate error message/s if the operation cannot be performed, otherwise print the result: "the sum of 'userv1' and 'userv2' is ...." . Execute the shell script as many number of times the user wants. Show all possible combinations of the types of variables and result/s of the corresponding operation/s.

SOURCE CODE :

```
#!/bin/bash

while true
do
    echo "Enter first value:"
    read userv1

    echo "Enter second value:"
    read userv2

    # Check if both are integers or real numbers
    if [[ $userv1 =~ ^-?[0-9]+([.][0-9]+)?$ && $userv2 =~ ^-?[0-
9]+([.][0-9]+)?$ ]]
    then
        sum=$(echo "$userv1 + $userv2" | bc)
        sub=$(echo "$userv1 - $userv2" | bc)
        mul=$(echo "$userv1 * $userv2" | bc)

        if [ "$(echo "$userv2 == 0" | bc)" -eq 1 ]
        then
            div="Undefined (Division by Zero)"
        else
            div=$(echo "scale=2; $userv1 / $userv2" | bc)
        fi

        echo "Sum        = $sum"
        echo "Difference = $sub"
        echo "Product    = $mul"
        echo "Division   = $div"

    else
        echo "Error: Arithmetic operations cannot be performed on
non-numeric values."
    fi

    echo
    echo "Do you want to continue? (y/n)"
```

```
    read ch

    if [ "$ch" != "y" ]
    then
        break
    fi
done
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q1.sh
Enter first value:
10
Enter second value:
20
Sum        = 30
Difference = -10
Product    = 200
Division   = .50

Do you want to continue? (y/n)
n
```

QUESTION 2: Write a shell script which accepts basic salary, House Rent allowance and Dearness Allowance of an employee and calculates net salary and displays the salary slip containing the Name of the Employee and the net salary.

SOURCE CODE :

```
#!/bin/bash

echo "Enter Employee Name:"
read name

echo "Enter Basic Salary:"
read basic

echo "Enter House Rent Allowance (HRA):"
read hra

echo "Enter Dearness Allowance (DA):"
read da

net=$(echo "$basic + $hra + $da" | bc)

echo
echo "========== Salary Slip =========="
echo "Employee Name : $name"
echo "Basic Salary  : $basic"
echo "HRA           : $hra"
echo "DA            : $da"
echo "Net Salary    : $net"
echo "================================="
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q2.sh
Enter Employee Name:
Snehasish Sarkar
Enter Basic Salary:
80000
Enter House Rent Allowance (HRA):
10000
Enter Dearness Allowance (DA):
20000

========== Salary Slip ==========
Employee Name : Snehasish Sarkar
Basic Salary  : 80000
HRA           : 10000
DA            : 20000
Net Salary    : 110000
=================================
```

QUESTION 3: Write a shell script which accepts a four digit number (integer) and a) displays the sum of its digits, b) displays the integer in reverse [that is, if you enter 7890, the program output will be 0987].

SOURCE CODE :

```
#!/bin/bash

echo "Enter a 4-digit number:"
read num

if [[ ! $num =~ ^[0-9]{4}$ ]]
then
    echo "Error: Please enter a valid 4-digit number."
    exit
fi

d1=$((num / 1000))
d2=$(((num / 100) % 10))
d3=$(((num / 10) % 10))
d4=$((num % 10))

sum=$((d1 + d2 + d3 + d4))

echo "Sum of digits = $sum"
echo "Reverse = $d4$d3$d2$d1"
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q3.sh
Enter a 4-digit number:
4563
Sum of digits = 18
Reverse = 3654
```

QUESTION 4: Write a shell script a) to find out the total count of files and directories in the current /any other directory. b) Also list the file and directory names separately with the respective counts. c) b) The script should also print out the total size of the files created in the past week.

SOURCE CODE :

```
#!/bin/bash

echo "Enter directory path (Press Enter for current directory):"
read dir

if [ -z "$dir" ]
then
    dir="."
fi

if [ ! -d "$dir" ]
then
    echo "Directory does not exist!"
    exit
fi

echo
echo "===== Files ====="
find "$dir" -maxdepth 1 -type f -printf "%f\n"
fileCount=$(find "$dir" -maxdepth 1 -type f | wc -l)
echo "Total Files: $fileCount"

echo
echo "===== Directories ====="
find "$dir" -maxdepth 1 -type d -printf "%f\n"
dirCount=$(find "$dir" -maxdepth 1 -type d | wc -l)
echo "Total Directories: $dirCount"

echo
echo "Total Files + Directories: $((fileCount + dirCount))"

echo
echo "Total size of files created/modified in the last 7 days:"
find "$dir" -maxdepth 1 -type f -mtime -7 -printf "%s\n" | \
awk '{sum += $1} END {print sum, "bytes"}'
```

OUTPUT :

```
Enter directory path (Press Enter for current directory):


===== Files =====
a3q8.c
a3q1.c
a4q1a.c
a2q4.c
a1q2.sh
a3q3.c
a2q2.c
a3q4.c
a2q3.c
a1q1.sh
a1q3.sh
a3q7.c
a.out
a3q2.c
a3q6.c
a2q1.c
a3q5.c
a4q2.c
a4q1b.c
a1q4.sh
Total Files: 20

===== Directories =====
.
Total Directories: 1

Total Files + Directories: 21

Total size of files created/modified in the last 7 days:
36167 bytes
```

QUESTION 5: Write a script that adds up the sizes reported by the ls command for the files in the
current / any other directory. a) The script should print out only the total number of bytes
used. b) The script should also print out the total size of the files created in the day that you are using this script.

SOURCE CODE :

```
#!/bin/bash

echo "Enter directory path (Press Enter for current directory):"
read dir

if [ -z "$dir" ]
then
    dir="."
fi

if [ ! -d "$dir" ]
then
    echo "Directory does not exist!"
    exit
fi

# (a) Total size of all files
total=$(find "$dir" -maxdepth 1 -type f -printf "%s\n" | \
awk '{sum += $1} END {print sum}')

echo "Total bytes used by all files: $total bytes"

# (b) Total size of today's files (modified today)
today=$(find "$dir" -maxdepth 1 -type f -daystart -mtime 0 -printf
"%s\n" | \
awk '{sum += $1} END {print sum}')

echo "Total size of files modified today: $today bytes"
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q5.sh
Enter directory path (Press Enter for current directory):

Total bytes used by all files: 36764 bytes
Total size of files modified today: 36764 bytes
```

QUESTION 6: The PATH variable is an environment variable that contains an ordered list of paths that
Linux will search for executables when running a command. Write a shell script to display all the directories in the PATH variable in a simple way, i.e., one line per directory. In addition, display information about each directory, such as the permissions
and the modification times.

SOURCE CODE :

```
#!/bin/bash

echo "Directories in PATH:"
echo "------------------------------"

echo $PATH | tr ":" "\n" | while read dir
do
    echo
    echo "Directory: $dir"

    if [ -d "$dir" ]
    then
        ls -ld "$dir"
    else
        echo "Directory does not exist."
    fi
done
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q6.sh
Directories in PATH:
-----------------------------

Directory: /home/hero/.local/bin
drwxr-xr-x 2 hero hero 4096 Feb 26  2025 /home/hero/.local/bin

Directory: /usr/local/sbin
drwxr-xr-x 2 root root 4096 Feb 26  2025 /usr/local/sbin

Directory: /usr/sbin
drwxr-xr-x 2 root root 20480 Jun 23  2025 /usr/sbin

Directory: /sbin
lrwxrwxrwx 1 root root 8 Feb 26  2025 /sbin -> usr/sbin

Directory: /usr/local/bin
drwxr-xr-x 2 root root 4096 Aug  8  2025 /usr/local/bin

Directory: /usr/bin
drwxr-xr-x 2 root root 126976 Jun 10 15:27 /usr/bin

Directory: /bin
lrwxrwxrwx 1 root root 7 Feb 26  2025 /bin -> usr/bin

Directory: /usr/local/games
drwxr-xr-x 2 root root 4096 Feb 26  2025 /usr/local/games

Directory: /usr/games
drwxr-xr-x 2 root root 4096 Nov 15  2024 /usr/games

Directory: /home/hero/.dotnet/tools
Directory does not exist.
```

QUESTION 7: Write a shell script which displays vendor id, model name, cpu MHz, cache size information about the processor present in your computer. [Hint: most of this information can be obtained by reading the file /proc/cpuinfo.]

SOURCE CODE :

```
#!/bin/bash

echo "===== CPU Information ====="

echo "Vendor ID :"
grep "vendor_id" /proc/cpuinfo | head -1

echo

echo "Model Name :"
grep "model name" /proc/cpuinfo | head -1

echo

echo "CPU MHz :"
grep "cpu MHz" /proc/cpuinfo | head -1

echo

echo "Cache Size :"
```

```
grep "cache size" /proc/cpuinfo | head -1
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q7.sh
===== CPU Information =====
Vendor ID :
vendor_id       : AuthenticAMD

Model Name :
model name      : AMD Ryzen 5 5600H with Radeon Graphics

CPU MHz :
cpu MHz         : 3293.814

Cache Size :
cache size      : 512 KB
```

QUESTION 8: Write a shell script multiplication_table that will generate a multiplication table for a number given on the command line.

SOURCE CODE :

```
#!/bin/bash

if [ $# -ne 1 ]
then
    echo "Usage: ./multiplication_table.sh <number>"
    exit 1
fi

num=$1

echo "Multiplication Table of $num"

for i in {1..10}
do
    echo "$num x $i = $((num * i))"
done
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q8.sh 10
Multiplication Table of 10
10 x 1 = 10
10 x 2 = 20
10 x 3 = 30
10 x 4 = 40
10 x 5 = 50
10 x 6 = 60
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
```

QUESTION 9: Write a shell script to count the number of lines in a file.

SOURCE CODE :

```
#!/bin/bash

if [ $# -ne 1 ]
then
    echo "Usage: ./count_lines.txt <filename>"
    exit 1
fi

if [ ! -f "$1" ]
then
    echo "File does not exist!"
    exit 1
fi

lines=$(wc -l < "$1")

echo "Number of lines in $1 = $lines"
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a1q9.sh count_lines.txt
Number of lines in count_lines.txt = 31
```

QUESTION 10: Design a CPU scheduler (choose any programming language of your preference) for jobs whose execution profiles will be in a file that is to be read and appropriate scheduling algorithm to be chosen by the scheduler.
Format of the profile:
<Job id> <priority> <arrival time> <CPU burst(1) I/O burst(1) CPU burst(2) ..... >-1
(Each information is separated by blank space and each job profile ends with -1. Lesser priority number denotes higher priority process with priority number 1 being the process with highest priority.) Example: 2 3 4 100 2 200 3 25 -1 7 1 8 60 10 ..... -1 etc.
Testing:

A. Create job profiles for 30 jobs and use three different scheduling algorithms (FCFS, Priority and Round Robin (?me slice: 15)). You may also test with other scheduling algorithms that you have done in class.

B. Compare the average waiting ?me, turnaround ?me of each process for the different scheduling algorithms.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 5
#define TQ 15

typedef struct
{
    int id;
    int priority;
    int arrival;
    int burst;

    int remaining;
    int waiting;
    int turnaround;
    int completed;
} Process;

void generate(Process p[])
{
    srand(time(NULL));

    printf("Generated Processes\n");
    printf("-----------------------------------------\n");
    printf("ID\tPri\tAT\tBT\n");

    for(int i=0;i<N;i++)
```

```
    {
        p[i].id=i+1;
        p[i].priority=1+rand()%5;
        p[i].arrival=rand()%20;
        p[i].burst=10+rand()%91;

        p[i].remaining=p[i].burst;
        p[i].completed=0;

        printf("%d\t%d\t%d\t%d\n",
               p[i].id,
               p[i].priority,
               p[i].arrival,
               p[i].burst);
    }

    printf("\n");
}

void reset(Process a[],Process b[])
{
    for(int i=0;i<N;i++)
    {
        b[i]=a[i];
        b[i].remaining=b[i].burst;
        b[i].completed=0;
    }
}

void FCFS(Process p[])
{
    int time=0;

    for(int i=0;i<N-1;i++)
        for(int j=i+1;j<N;j++)
            if(p[i].arrival>p[j].arrival)
            {
                Process t=p[i];
                p[i]=p[j];
                p[j]=t;
            }

    double aw=0,at=0;

    printf("\n========== FCFS ==========\n");
```

```
    printf("ID\tWT\tTAT\n");

    for(int i=0;i<N;i++)
    {
        if(time<p[i].arrival)
            time=p[i].arrival;

        p[i].waiting=time-p[i].arrival;

        time+=p[i].burst;

        p[i].turnaround=time-p[i].arrival;

        aw+=p[i].waiting;
        at+=p[i].turnaround;

        printf("%d\t%d\t%d\n",
               p[i].id,
               p[i].waiting,
               p[i].turnaround);
    }

    printf("Average Waiting Time = %.2lf\n",aw/N);
    printf("Average Turnaround Time = %.2lf\n",at/N);
}

void Priority(Process p[])
{
    int done=0,time=0;

    double aw=0,at=0;

    printf("\n========== PRIORITY ==========\n");
    printf("ID\tWT\tTAT\n");

    while(done<N)
    {
        int idx=-1;

        for(int i=0;i<N;i++)
        {
            if(!p[i].completed && p[i].arrival<=time)
            {
                if(idx==-1 || p[i].priority<p[idx].priority)
                    idx=i;
            }
```

```
        }

        if(idx==-1)
        {
            time++;
            continue;
        }

        p[idx].waiting=time-p[idx].arrival;

        time+=p[idx].burst;

        p[idx].turnaround=time-p[idx].arrival;

        p[idx].completed=1;

        done++;

        aw+=p[idx].waiting;
        at+=p[idx].turnaround;

        printf("%d\t%d\t%d\n",
               p[idx].id,
               p[idx].waiting,
               p[idx].turnaround);
    }

    printf("Average Waiting Time = %.2lf\n",aw/N);
    printf("Average Turnaround Time = %.2lf\n",at/N);
}

void RoundRobin(Process p[])
{
    int queue[500];
    int front=0,rear=0;

    int visited[N]={0};

    int completed=0;
    int time=0;

    double aw=0,at=0;

    printf("\n========== ROUND ROBIN (TQ=15) ==========\n");

    while(completed<N)
```

```
{
    for(int i=0;i<N;i++)
    {
        if(!visited[i] && p[i].arrival<=time)
        {
            queue[rear++]=i;
            visited[i]=1;
        }
    }

    if(front==rear)
    {
        time++;
        continue;
    }

    int i=queue[front++];

    if(p[i].remaining>TQ)
    {
        p[i].remaining-=TQ;
        time+=TQ;

        for(int j=0;j<N;j++)
        {
            if(!visited[j] && p[j].arrival<=time)
            {
                queue[rear++]=j;
                visited[j]=1;
            }
        }

        queue[rear++]=i;
    }
    else
    {
        time+=p[i].remaining;
        p[i].remaining=0;

        p[i].turnaround=time-p[i].arrival;
        p[i].waiting=p[i].turnaround-p[i].burst;

        completed++;

        aw+=p[i].waiting;
        at+=p[i].turnaround;
```

```
        }
    }

    printf("ID\tWT\tTAT\n");

    for(int i=0;i<N;i++)
    {
        printf("%d\t%d\t%d\n",
               p[i].id,
               p[i].waiting,
               p[i].turnaround);
    }

    printf("Average Waiting Time = %.2lf\n",aw/N);
    printf("Average Turnaround Time = %.2lf\n",at/N);
}

int main()
{
    Process original[N],temp[N];

    generate(original);

    reset(original,temp);
    FCFS(temp);

    reset(original,temp);
    Priority(temp);

    reset(original,temp);
    RoundRobin(temp);

    return 0;
}
```

OUTPUT :

```
$ ./a
Generated Processes
-----------------------------------------
ID      Pri     AT      BT
1       1       5       47
2       1       19      26
3       2       3       38
4       5       0       18
5       3       17      64


========== FCFS ==========
ID      WT      TAT
4       0       18
3       15      53
1       51      98
5       86      150
2       148     174
Average Waiting Time = 60.00
Average Turnaround Time = 98.60

========== PRIORITY ==========
ID      WT      TAT
4       0       18
1       13      60
2       46      72
3       88      126
5       112     176
Average Waiting Time = 51.80
Average Turnaround Time = 90.40

========== ROUND ROBIN (TQ=15) ==========
ID      WT      TAT
1       122     169
2       74      100
3       116     154
4       30      48
5       112     176
Average Waiting Time = 90.80
Average Turnaround Time = 129.40
```

QUESTION 11 : Write a C program to create a child process. The parent process must wait until the child finishes. Both the processes must print their own pid and parent pid. Additionally the parent process should print the exit status of the child.

SOURCE CODE :

```
#include <stdio.h>
#include <windows.h>

int main(int argc, char *argv[])
{
    // Child process
    if (argc > 1)
    {
        printf("----- Child Process -----\n");
        printf("Child PID  : %lu\n", GetCurrentProcessId());
        printf("Parent PID : Not directly available in Windows\n");

        return 10;   // Exit status
    }

    // Parent process
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    char cmd[] = "child.exe child";

    if (!CreateProcess(
            NULL,
            cmd,
            NULL,
            NULL,
            FALSE,
            0,
            NULL,
            NULL,
            &si,
            &pi))
    {
        printf("CreateProcess failed (%lu)\n", GetLastError());
        return 1;
    }
```

```
    // Wait for child
    WaitForSingleObject(pi.hProcess, INFINITE);

    DWORD exitCode;
    GetExitCodeProcess(pi.hProcess, &exitCode);

    printf("\n----- Parent Process -----\n");
    printf("Parent PID : %lu\n", GetCurrentProcessId());
    printf("Child Exit Status : %lu\n", exitCode);

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out

--- Child Process ---
Child PID  : 3328
Parent PID : 3327
Child is executing...

--- Parent Process ---
Parent PID : 3327
Parent's Parent PID : 2924
Child Exit Status : 10
```

QUESTION 12: Write a C program which prints prime numbers between the range 1 to 10,00,000 by creating ten child processes and subdividing the task equally among all child processes, i.e., the first child should print prime numbers in the range 1 to 1,00,000, the second child in the range 1,00,001 to 2,00,000, ... The child processes must run in parallel and the parent process must wait until all the child processes finish.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define LIMIT 1000000
#define CHILDREN 10

int isPrime(int n)
{
    if (n < 2)
        return 0;

    for (int i = 2; i * i <= n; i++)
        if (n % i == 0)
            return 0;

    return 1;
}

int main()
{
    int range = LIMIT / CHILDREN;

    for (int i = 0; i < CHILDREN; i++)
    {
        pid_t pid = fork();

        if (pid == 0)
        {
            int start = i * range + 1;
            int end = (i + 1) * range;
            int count = 0;

            for (int j = start; j <= end; j++)
                if (isPrime(j))
                    count++;
```

```
            printf("Child %2d | PID: %5d | Range: %6d - %7d |
Primes: %d\n",
                   i + 1, getpid(), start, end, count);

            exit(0);
        }
    }

    // Parent waits for all children
    for (int i = 0; i < CHILDREN; i++)
        wait(NULL);

    printf("\nParent: All child processes have finished.\n");

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Child  1 | PID:  5785 | Range:      1 -  100000 | Primes: 9592
Child  5 | PID:  5789 | Range: 400001 -  500000 | Primes: 7678
Child  8 | PID:  5792 | Range: 700001 -  800000 | Primes: 7408
Child  3 | PID:  5787 | Range: 200001 -  300000 | Primes: 8013
Child  4 | PID:  5788 | Range: 300001 -  400000 | Primes: 7863
Child  6 | PID:  5790 | Range: 500001 -  600000 | Primes: 7560
Child  2 | PID:  5786 | Range: 100001 -  200000 | Primes: 8392
Child  7 | PID:  5791 | Range: 600001 -  700000 | Primes: 7445
Child  9 | PID:  5793 | Range: 800001 -  900000 | Primes: 7323
Child 10 | PID:  5794 | Range: 900001 - 1000000 | Primes: 7224

Parent: All child processes have finished.
```

QUESTION 13 : Write a C program which creates a child process. The parent process sends a string (input by user) which the child process inspects and sends ”YES” back to the parent if the string is a palindrome, otherwise it sends ”NO”. The IPC to be used is pipe. Both the processes terminate when the input string is “quit”.

SOURCE CODE :

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int isPalindrome(char str[])
{
    int i = 0;
    int j = strlen(str) - 1;

    while (i < j)
    {
        if (str[i] != str[j])
            return 0;
        i++;
        j--;
    }
    return 1;
}

int main()
{
    int p1[2], p2[2];
    pipe(p1);   // Parent -> Child
    pipe(p2);   // Child -> Parent

    pid_t pid = fork();

    if (pid == 0)
    {
        // Child
        char str[100], result[10];

        while (1)
        {
            read(p1[0], str, sizeof(str));

            if (strcmp(str, "quit") == 0)
                break;
```

```
            if (isPalindrome(str))
                strcpy(result, "YES");
            else
                strcpy(result, "NO");

            write(p2[1], result, sizeof(result));
        }
    }
    else
    {
        // Parent
        char str[100], result[10];

        while (1)
        {
            printf("Enter string: ");
            scanf("%s", str);

            write(p1[1], str, sizeof(str));

            if (strcmp(str, "quit") == 0)
                break;

            read(p2[0], result, sizeof(result));
            printf("Child says: %s\n\n", result);
        }

        wait(NULL);
    }

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Enter string: hello
Child says: NO

Enter string: level
Child says: YES

Enter string: madam
Child says: YES

Enter string: abc
Child says: NO

Enter string: quit
```

QUESTION 14: Write a program for the Reader-Writer process for the following situation:
a) Multiple readers and one writer: writer gets to write whenever it is ready (reader/s wait)

SOURCE CODE:

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int readCount = 0;
int data = 0;

sem_t mutex;       // Protects readCount
sem_t wrt;         // Controls database access
sem_t readTry;     // Blocks readers when writer is waiting

void *reader(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&readTry);
    sem_wait(&mutex);

    readCount++;
    if (readCount == 1)
        sem_wait(&wrt);

    sem_post(&mutex);
    sem_post(&readTry);

    printf("Reader %d is reading: %d\n", id, data);
```

```
    sleep(1);

    sem_wait(&mutex);

    readCount--;
    if (readCount == 0)
        sem_post(&wrt);

    sem_post(&mutex);

    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&readTry);   // Stop new readers
    sem_wait(&wrt);

    data++;
    printf("Writer %d is writing: %d\n", id, data);
    sleep(2);

    sem_post(&wrt);
    sem_post(&readTry);

    return NULL;
}

int main()
{
    pthread_t r[3], w;
    int rid[3] = {1, 2, 3};
    int wid = 1;

    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);
    sem_init(&readTry, 0, 1);
    pthread_create(&r[0], NULL, reader, &rid[0]);
    pthread_create(&r[1], NULL, reader, &rid[1]);
    sleep(1);
    pthread_create(&w, NULL, writer, &wid);
    sleep(1);
    pthread_create(&r[2], NULL, reader, &rid[2]);
    for (int i = 0; i < 3; i++)
```

```
        pthread_join(r[i], NULL);

    pthread_join(w, NULL);
    sem_destroy(&mutex);
    sem_destroy(&wrt);
    sem_destroy(&readTry);
    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Reader 1 is reading: 0
Reader 2 is reading: 0
Writer 1 is writing: 1
Reader 3 is reading: 1
```

QUESTION 15: Write a C program to simulate Bankers Algorithm for Deadlock Avoidance.

SOURCE CODE:

```
#include <stdio.h>

#define P 5
#define R 3

int main()
{
    int alloc[P][R] = {
        {0,1,0},
        {2,0,0},
        {3,0,2},
        {2,1,1},
        {0,0,2}
    };

    int max[P][R] = {
        {7,5,3},
        {3,2,2},
        {9,0,2},
        {2,2,2},
        {4,3,3}
    };

    int avail[R] = {3,3,2};

    int need[P][R];
    int finish[P] = {0};
    int safe[P];
```

```
// Calculate Need Matrix
for(int i=0;i<P;i++)
    for(int j=0;j<R;j++)
        need[i][j] = max[i][j] - alloc[i][j];

int count = 0;

while(count < P)
{
    int found = 0;

    for(int i=0;i<P;i++)
    {
        if(!finish[i])
        {
            int j;

            for(j=0;j<R;j++)
                if(need[i][j] > avail[j])
                    break;

            if(j==R)
            {
                for(int k=0;k<R;k++)
                    avail[k] += alloc[i][k];

                finish[i] = 1;
                safe[count++] = i;
                found = 1;
            }
        }
    }

    if(!found)
        break;
}

if(count == P)
{
    printf("System is in SAFE state.\n");
    printf("Safe Sequence: ");

    for(int i=0;i<P;i++)
        printf("P%d ", safe[i]);
```

```
        printf("\n");
    }
    else
    {
        printf("System is NOT in SAFE state.\n");
    }

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
```

QUESTION 16: Write a C program to simulate Worst-Fit memory allocation technique.

SOURCE CODE:

```
#include <stdio.h>

#define BLOCKS 5
#define PROCESSES 4

int main()
{
    // Static Input
    int blockSize[BLOCKS] = {100, 500, 200, 300, 600};
    int processSize[PROCESSES] = {212, 417, 112, 426};

    int allocation[PROCESSES];

    for (int i = 0; i < PROCESSES; i++)
        allocation[i] = -1;

    // Worst-Fit Allocation
    for (int i = 0; i < PROCESSES; i++)
    {
        int worst = -1;

        for (int j = 0; j < BLOCKS; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worst == -1 || blockSize[j] > blockSize[worst])
                    worst = j;
```

```
            }
        }

        if (worst != -1)
        {
            allocation[i] = worst;
            blockSize[worst] -= processSize[i];
        }
    }

    // Output
    printf("Process\tSize\tBlock\n");

    for (int i = 0; i < PROCESSES; i++)
    {
        printf("P%d\t%d\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Process Size    Block
P1      212     5
P2      417     2
P3      112     5
P4      426     Not Allocated
```

QUESTION 17: Write a C program to simulate First-Fit memory allocation technique.
SOURCE CODE:

```c
#include <stdio.h>

#define BLOCKS 5
#define PROCESSES 4

int main()
{
    int blockSize[BLOCKS] = {100, 500, 200, 300, 600};
    int processSize[PROCESSES] = {212, 417, 112, 426};
    int allocation[PROCESSES];

    for (int i = 0; i < PROCESSES; i++)
        allocation[i] = -1;
    for (int i = 0; i < PROCESSES; i++)
    {
        for (int j = 0; j < BLOCKS; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
    printf("Process\tSize\tBlock\n");
    for (int i = 0; i < PROCESSES; i++)
    {
        printf("P%d\t%d\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Process Size    Block
P1      212     2
P2      417     5
P3      112     2
P4      426     Not Allocated
```

QUESTION 18: Write a C program for simulating the Indexed File Allocation algorithm.

SOURCE CODE:

```c
#include <stdio.h>

#define TOTAL_BLOCKS 20

int main()
{
    // 0 = Free, 1 = Allocated
    int memory[TOTAL_BLOCKS] = {0};

    // Static Input
    int indexBlock = 5;
    int fileBlocks[] = {2, 7, 10, 15};
    int n = 4;

    // Check if index block is free
    if (memory[indexBlock] == 1)
    {
        printf("Index Block %d is already allocated.\n",
indexBlock);
        return 0;
    }

    memory[indexBlock] = 1;

    // Allocate data blocks
    for (int i = 0; i < n; i++)
    {
        if (memory[fileBlocks[i]] == 0)
            memory[fileBlocks[i]] = 1;
        else
        {
            printf("Block %d is already allocated.\n",
fileBlocks[i]);
            return 0;
        }
    }
```

```
    // Display Allocation
    printf("Indexed File Allocation\n");
    printf("-----------------------\n");
    printf("Index Block : %d\n", indexBlock);

    printf("Data Blocks : ");
    for (int i = 0; i < n; i++)
        printf("%d ", fileBlocks[i]);

    printf("\n");

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Indexed File Allocation
-----------------------
Index Block : 5
Data Blocks : 2 7 10 15
```

QUESTION 19: Write a C program to simulate FCFS (First Come First Serve) Disk scheduling algorithm.

SOURCE CODE:

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // Static Input
    int request[] = {98, 183, 37, 122, 14, 124, 65, 67};
    int n = 8;
    int head = 53;

    int totalSeek = 0;

    printf("FCFS Disk Scheduling\n");
    printf("--------------------\n");
    printf("Initial Head Position = %d\n\n", head);

    printf("Head Movement:\n");
    printf("%d", head);

    for(int i = 0; i < n; i++)
    {
        totalSeek += abs(request[i] - head);
        head = request[i];
        printf(" -> %d", head);
    }

    printf("\n\nTotal Seek Time = %d\n", totalSeek);
    printf("Average Seek Time = %.2f\n", (float)totalSeek / n);

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
FCFS Disk Scheduling
--------------------
Initial Head Position = 53

Head Movement:
53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67

Total Seek Time = 640
Average Seek Time = 80.00
```

QUESTION 20: Write a program to simulate Least Recently Used (LRU) page replacement algorithm for the following page reference string: 9, 8, 11, 7, 12, 8, 7, 6, 8, 12, 5, 4, 12, 3, 10, 11, 12, 4, 9, 4. Consider (i) 4 frames and (ii) 5 frames. Compare the results.

SOURCE CODE:

```c
#include <stdio.h>

#define N 20

int pages[N] = {
    9, 8, 11, 7, 12, 8, 7, 6, 8, 12,
    5, 4, 12, 3, 10, 11, 12, 4, 9, 4
};

void LRU(int frames)
{
    int frame[10];
    int lastUsed[10];
    int faults = 0;

    for (int i = 0; i < frames; i++)
    {
        frame[i] = -1;
        lastUsed[i] = -1;
    }

    for (int i = 0; i < N; i++)
    {
        int found = 0;

        // Check if page is already present
        for (int j = 0; j < frames; j++)
        {
            if (frame[j] == pages[i])
            {
```

```
                found = 1;
                lastUsed[j] = i;
                break;
            }
        }

        // Page Fault
        if (!found)
        {
            int pos = 0;

            // Find empty frame
            for (int j = 0; j < frames; j++)
            {
                if (frame[j] == -1)
                {
                    pos = j;
                    goto allocate;
                }
            }

            pos = 0;
            for (int j = 1; j < frames; j++)
            {
                if (lastUsed[j] < lastUsed[pos])
                    pos = j;
            }

allocate:
            frame[pos] = pages[i];
            lastUsed[pos] = i;
            faults++;
        }
    }

    printf("Frames = %d\n", frames);
    printf("Total Page Faults = %d\n\n", faults);
}

int main()
{
    printf("LRU Page Replacement\n");
    printf("--------------------\n\n");

    LRU(4);
    LRU(5);
```

```
    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
LRU Page Replacement
--------------------

Frames = 4
Total Page Faults = 13

Frames = 5
Total Page Faults = 12
```

QUESTION 21: Write a program to simulate First In First Out (FIFO) page replacement algorithm for the following page reference string: 9, 10, 7, 12, 7, 6, 12, 6, 5, 4, 3, 10, 11, 12, 4, 5, 6, 9, 4, 5. Consider (i) 3 frames and (ii) 4 frames. Compare the results.

SOURCE CODE:

```
#include <stdio.h>

#define N 20

int pages[N] = {
    9, 10, 7, 12, 7, 6, 12, 6, 5, 4,
    3, 10, 11, 12, 4, 5, 6, 9, 4, 5
};

void FIFO(int frames)
{
    int frame[10];
    int faults = 0;
    int index = 0;

    for (int i = 0; i < frames; i++)
        frame[i] = -1;

    for (int i = 0; i < N; i++)
    {
        int found = 0;

        // Check if page already exists
        for (int j = 0; j < frames; j++)
        {
            if (frame[j] == pages[i])
            {
                found = 1;
```

```
                break;
            }
        }

        // Page Fault
        if (!found)
        {
            frame[index] = pages[i];
            index = (index + 1) % frames;
            faults++;
        }
    }

    printf("Frames = %d\n", frames);
    printf("Total Page Faults = %d\n\n", faults);
}

int main()
{
    printf("FIFO Page Replacement\n");
    printf("---------------------\n\n");

    FIFO(3);
    FIFO(4);

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
FIFO Page Replacement
---------------------

Frames = 3
Total Page Faults = 17

Frames = 4
Total Page Faults = 15
```

QUESTION 22 : Write a program for the Reader-Writer process for the following situation:

a) Multiple readers and multiple writers: writers have priority

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int data = 0, readCount = 0;

sem_t mutex, wrt, readTry;

void *reader(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&readTry);
    sem_wait(&mutex);

    readCount++;
    if(readCount == 1)
        sem_wait(&wrt);

    sem_post(&mutex);
    sem_post(&readTry);

    printf("Reader %d reads %d\n", id, data);
    sleep(1);

    sem_wait(&mutex);
    readCount--;

    if(readCount == 0)
        sem_post(&wrt);

    sem_post(&mutex);

    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&readTry);
```

```
    sem_wait(&wrt);

    data++;
    printf("Writer %d writes %d\n", id, data);
    sleep(1);

    sem_post(&wrt);
    sem_post(&readTry);

    return NULL;
}

int main()
{
    pthread_t r[3], w[2];

    int rid[3]={1,2,3};
    int wid[2]={1,2};

    sem_init(&mutex,0,1);
    sem_init(&wrt,0,1);
    sem_init(&readTry,0,1);

    pthread_create(&r[0],NULL,reader,&rid[0]);
    pthread_create(&r[1],NULL,reader,&rid[1]);

    pthread_create(&w[0],NULL,writer,&wid[0]);

    pthread_create(&r[2],NULL,reader,&rid[2]);

    pthread_create(&w[1],NULL,writer,&wid[1]);

    for(int i=0;i<3;i++)
        pthread_join(r[i],NULL);

    for(int i=0;i<2;i++)
        pthread_join(w[i],NULL);

    return 0;
}
```

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Reader 1 reads 0
Reader 2 reads 0
Writer 1 writes 1
Reader 3 reads 1
Writer 2 writes 2
```

b) Multiple readers and multiple writers: readers have priority

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

int data = 0, readCount = 0;

sem_t mutex, wrt;

void *reader(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&mutex);

    readCount++;
    if(readCount == 1)
        sem_wait(&wrt);

    sem_post(&mutex);

    printf("Reader %d reads %d\n", id, data);
    sleep(1);

    sem_wait(&mutex);

    readCount--;
    if(readCount == 0)
        sem_post(&wrt);

    sem_post(&mutex);

    return NULL;
}

void *writer(void *arg)
{
```

```
    int id = *(int *)arg;

    sem_wait(&wrt);

    data++;
    printf("Writer %d writes %d\n", id, data);
    sleep(1);

    sem_post(&wrt);

    return NULL;
}

int main()
{
    pthread_t r[3], w[2];

    int rid[3]={1,2,3};
    int wid[2]={1,2};

    sem_init(&mutex,0,1);
    sem_init(&wrt,0,1);

    pthread_create(&r[0],NULL,reader,&rid[0]);
    pthread_create(&r[1],NULL,reader,&rid[1]);

    pthread_create(&w[0],NULL,writer,&wid[0]);

    pthread_create(&r[2],NULL,reader,&rid[2]);

    pthread_create(&w[1],NULL,writer,&wid[1]);

    for(int i=0;i<3;i++)
        pthread_join(r[i],NULL);

    for(int i=0;i<2;i++)
        pthread_join(w[i],NULL);

    return 0;
}
```

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Reader 1 reads 0
Reader 2 reads 0
Reader 3 reads 0
Writer 1 writes 1
Writer 2 writes 2
```

QUESTION 23: Write a C program to simulate SCAN and CSCAN Disk scheduling algorithm.

a) Use 3 randomly generated lists of disk requests spanning from cylinders 0 (outermost) to 199 (innermost)
b) Test each list with each disk scheduling algorithm and compare the results
c) Generate a list of requests in which the requests are concentrated at one end of the disk
d) Generate a list of request in which the requests are concentrated at the centre of the disk
Repeat step (b) for both (c) and (d).

SOURCE CODE :

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 10
#define MAX 199

void sort(int a[], int n)
{
    for(int i=0;i<n-1;i++)
        for(int j=0;j<n-i-1;j++)
            if(a[j]>a[j+1])
            {
                int t=a[j];
                a[j]=a[j+1];
                a[j+1]=t;
            }
}

void SCAN(int req[], int n, int head)
{
```

```
    int a[20];

    for(int i=0;i<n;i++)
        a[i]=req[i];

    sort(a,n);

    int pos=0;

    while(pos<n && a[pos]<head)
        pos++;

    int seek=0,temp=head;

    // Move towards higher cylinders
    for(int i=pos;i<n;i++)
    {
        seek+=abs(a[i]-temp);
        temp=a[i];
    }

    if(temp!=MAX)
    {
        seek+=abs(MAX-temp);
        temp=MAX;
    }

    // Reverse direction
    for(int i=pos-1;i>=0;i--)
    {
        seek+=abs(temp-a[i]);
        temp=a[i];
    }

    printf("SCAN  : Total Seek=%d  Avg=%.2f\n",
            seek,(float)seek/n);
}

void CSCAN(int req[], int n, int head)
{
    int a[20];

    for(int i=0;i<n;i++)
        a[i]=req[i];

    sort(a,n);
```

```
    int pos=0;

    while(pos<n && a[pos]<head)
        pos++;

    int seek=0,temp=head;

    for(int i=pos;i<n;i++)
    {
        seek+=abs(a[i]-temp);
        temp=a[i];
    }

    if(temp!=MAX)
    {
        seek+=MAX-temp;
        temp=MAX;
    }

    seek+=MAX;
    temp=0;

    for(int i=0;i<pos;i++)
    {
        seek+=abs(a[i]-temp);
        temp=a[i];
    }

    printf("CSCAN : Total Seek=%d  Avg=%.2f\n",
            seek,(float)seek/n);
}

void randomList(int a[])
{
    for(int i=0;i<N;i++)
        a[i]=rand()%200;
}

void endList(int a[])
{
    for(int i=0;i<N;i++)
        a[i]=rand()%40;
}

void centerList(int a[])
```

```
{
    for(int i=0;i<N;i++)
        a[i]=80+rand()%41;
}

void printList(int a[])
{
    for(int i=0;i<N;i++)
        printf("%d ",a[i]);
    printf("\n");
}

int main()
{
    srand(time(NULL));

    int req[N];
    int head=100;

    printf("Initial Head = %d\n\n",head);

    // Three Random Lists
    for(int i=1;i<=3;i++)
    {
        printf("Random List %d:\n",i);

        randomList(req);

        printList(req);

        SCAN(req,N,head);
        CSCAN(req,N,head);

        printf("\n");
    }

    // One End
    printf("Requests Near One End:\n");

    endList(req);

    printList(req);

    SCAN(req,N,head);
    CSCAN(req,N,head);
```

```
    printf("\n");

    // Centre
    printf("Requests Near Centre:\n");

    centerList(req);

    printList(req);

    SCAN(req,N,head);
    CSCAN(req,N,head);

    return 0;
}
```

OUTPUT :

```
┌──(hero㉿hero)-[~/Desktop/OS]
└─$ ./a.out
Initial Head = 100

Random List 1:
138 32 117 43 69 21 97 131 75 43
SCAN  : Total Seek=277  Avg=27.70
CSCAN : Total Seek=395  Avg=39.50

Random List 2:
191 85 80 78 164 144 36 2 191 97
SCAN  : Total Seek=296  Avg=29.60
CSCAN : Total Seek=395  Avg=39.50

Random List 3:
68 138 12 171 176 50 195 158 85 66
SCAN  : Total Seek=286  Avg=28.60
CSCAN : Total Seek=383  Avg=38.30

Requests Near One End:
0 23 10 29 18 0 11 36 11 38
SCAN  : Total Seek=298  Avg=29.80
CSCAN : Total Seek=336  Avg=33.60

Requests Near Centre:
106 87 89 99 119 90 103 113 89 88
SCAN  : Total Seek=211  Avg=21.10
CSCAN : Total Seek=397  Avg=39.70
```